

# RetroEmu

*Summary: You've probably played on an emulator at least once to revisit the classic 8-bit handheld games of decades past. Today it's time to learn how emulation works and develop your own emulator for an 8-bit handheld console.*

*Version: 9.1*

# Contents


**I Forewords** **2**

**II Introduction** **3**

**III Objectives** **4**

**IV General instructions** **5**

**V Mandatory part** **6**
- V.1 CPU . . . 6
- V.2 LCD . . . 6
- V.3 Running thread and timing . . . 6
- V.4 Inputs . . . 6
- V.5 MBCs . . . 7
- V.6 CGB . . . 7

**VI Bonus part** **8**

**VII Submission and peer-evaluation** **10**

**VIII Credits** **11**

# Chapter I

# Forewords

Emulation is a long-standing tradition in computer science: rebuilding a piece of hardware in software, instruction by instruction, register by register, until vintage software runs again on machines for which it was never designed.

In this project you will tackle one of the canonical exercises of the field: emulating an 8-bit handheld console released in 1989, along with its color successor released in 1998. Both share the same CPU family (a custom 8-bit chip closely related to the Z80) and a similar display pipeline, which makes them a perfect playground to learn how a real piece of hardware works under the hood.

# Chapter II

# Introduction

This subject aims to make you build an emulator for an 8-bit handheld console (DMG model, released 1989) and its color variant (CGB model, released 1998). To do this you will need to correctly reproduce the behavior of the underlying hardware (CPU, PPU, MMU, cartridge mapping) using publicly available technical documentation maintained by the homebrew community (Pan Docs, GBDev wiki, etc.).

A video game console emulator is a type of emulator that allows a computing device, usually a personal computer, but also other video game consoles and mobile devices running operating systems such as Android to emulate a video game console's hardware and behavior and play games for that platform. Emulators are most often used to play older video games on personal computers and video game consoles, but they are also used to play games translated into other languages or to modify (or hack) existing games. More often than not, emulators offer additional features above and beyond that of the original console, such as multi-controller compatibility (such as PSX controllers being used with N64 games and vice versa), timescale control, higher framerates, higher resolutions, unlocking of gameplay features, memory modifications (like GameShark), and one-click cheat codes. Emulators are also a useful tool in the development process of homebrew demos and the creation of new games for older or discontinued consoles.

Code and data of a game are typically supplied to the emulator by means of a ROM file (a copy of the data contained on a game cartridge) or an ISO image (for systems that use optical media). Most game titles retain their copyright even with the original system and games being many years past discontinuation and increasing rarity. To test your emulator, you are expected to use legally-obtained ROMs only: homebrew releases, public test suites (e.g. Blargg's or Mooneye-GB test ROMs), or ROMs you have dumped from cartridges you own. Specialized adapters such as the Retrode allow emulators to directly access the data on game cartridges without the need to copy it into a ROM image first.

# Chapter III

# Objectives

This project has severals goals :

- Develop a real application from end to end
- Discover the principles of emulation
- Learn to read electronic documentation
- Play games from your childhood!

# Chapter IV

# General instructions

- This project will be evaluated only by humans.
- You must have a GUI with at minimum: load, play, pause.
- The project must build and run on Linux. It will be compiled on your machine and evaluated on the corrector's machine, so its build process must be reproducible.
- **Language:** you may write your emulator in either C++ or Rust. No version is imposed: pick whichever is supported by the toolchain available on the corrector's machine.
- **Imposed dependencies:** the graphics, input and audio layer must go through SDL2 (or, in Rust, an SDL2 binding such as the `sdl2` crate). No higher-level framework (Qt, GTK, SFML, wgpu, etc.) is allowed for the rendering layer.
- **Build system:** CMake for C++, Cargo for Rust. A single command must produce a single runnable executable.
- **No coding style is enforced.** You may follow any convention you like as long as your code remains readable to your peer evaluators.
- Provide a clear build procedure (e.g. a documented `cmake` invocation or `cargo build` target) and a single executable that launches the emulator. Your build output must bundle or document its runtime dependencies so the corrector can run it without manual setup.
- No commercial ROM may be committed to your repository. You are responsible for using only legally-obtained ROMs (homebrew, public test suites, or dumps of your own cartridges).

# Chapter V

# Mandatory part

## V.1 CPU

The basic hardware emulation of the handheld console starts with a good implementation of the CPU, ie the registers and instructions. You must also have a debugger allowing at least to:

- display the status of the registers
- display the next instruction to execute
- execute an instruction

## V.2 LCD

What would a handheld console be without a display? Not a great console!
You must correctly implement the display system (PPU): background, sprites, window, and palettes. The debugger will execute a single frame and/or one second of emulation in order to prove that there are visible changes on the screen.

## V.3 Running thread and timing

Now that the CPU and display are in place, it's time to make it all work together. Your emulator will run at normal speed without adversely affect the operations of the GUI.

## V.4 Inputs

Playing the intro is nice, actually playing the game is better!
You must correctly implement the input system of the handheld console: control pad (4 directions) and 4 action buttons (A, B, Start, Select).

## V.5 MBCs

In addition to ROMs only, you will discover that there are several types of cartridges with different characteristics. You will need to correctly implement the following memory bank controllers:

- MBC1 (with optional RAM and battery)
- MBC2
- MBC5 (with optional RAM and battery)

You must also manage the in-game backup for games that propose this feature (battery-backed cartridge RAM persisted on disk between sessions).

## V.6 CGB

Black and white was cool until 1967!
Now that the original (DMG) model has no secrets for you, it is time to move on to the color variant (CGB). These two machines are quite similar with a few notable differences (extended palette, double-speed mode, additional VRAM bank, HDMA). You must implement both.

# Chapter VI

# Bonus part

For this project, here is the list of the bonuses that will be counted. Each bonus is graded independently and awards a fixed number of points, up to a total of 25 bonus points: the Sound bonus is worth 5 points, and each of the other bonuses is worth 4 points.

- Sound :
  As part of this bonus, you have the right to use a library to manage the sound as the emulation of a synthesizer is not the objective of this subject.
- Boot sequence :
  Reproduce the console's power-on boot sequence and the correct post-boot register state for both the DMG and CGB models. The original boot ROM is copyrighted (it embeds the protected scrolling logo) and must not be shipped: write your own boot behaviour, use an open-source boot ROM (e.g. the MIT-licensed SameBoy one) or design a custom boot image.
- MBC3 (with RTC and battery) :
  In addition to the mandatory MBC1, MBC2 and MBC5, implement the MBC3 controller and its Real Time Clock, with the clock state persisted on disk between sessions. You will need a legally-obtained MBC3+RTC game to test it.
- Forcing DMG/CGB :
  You must be able to force the emulated model so you can launch a DMG game in color on the CGB and vice versa.
- Save States :
  You must be able to save the state of your game to resume it later.
- UX :
  Lots of super cool features enriching the user experience. examples:
  - reset
  - drag&drop
  - speed
  - etc.



The bonus part will only be assessed if the mandatory part is PERFECT. Perfect means the mandatory part has been integrally done and works without malfunctioning. If you have not passed ALL the mandatory requirements, your bonus part will not be evaluated at all.

# Chapter VII

# Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double check the names of your folders and files to ensure they are correct.

- A bundle of MIT-licensed test ROMs is provided at `attachments/test-roms.zip`. It is the canonical test material used during evaluation: extract it next to your executable and refer to its paths (e.g. `acid2/dmg-acid2.gb`, `mooneye/mbc5/rom_2Mb.gb`) when validating each item of the mandatory part.
- No commercial ROM may be committed to your repository. The bundle above is the only ROM material that will be evaluated. Any commercial ROM dumped, downloaded or otherwise obtained may be used by the team for personal exploration but must stay out of the repository.
- The project will be compiled on your machine and evaluated on the corrector's machine. Provide a clear build procedure (Makefile target or documented script) and a single launch command for the resulting executable.
- Your build output must bundle or document its runtime dependencies so the corrector can run the emulator without any manual setup beyond the documented build step.

# Chapter VIII

# Credits

★ ✧ ★

## A Word of Thanks

This project was born as a beta-test: a first version, submitted to a group of students before being made official for everyone.
We would like to warmly thank every student who took the time to work on it during that beta-test phase, and, most importantly, to report typos, inconsistencies, bugs, and suggestions along the way.
Their feedback made this subject clearer, fairer, and more solid. This project would not be what it is today without them.

☆ **Thank you!** ☆